

combine QSqlTableModel with List View, QDirModel with Tree View, etc.

The following code snippets illustrate the List View and Tree View, assuming two widgets by the object names `lv` and `tv` are added to the UI form:

```
QSqlList qsl = db.tables(); // db is connection object returned after
establishment
QSqlListModel *qsm1=new QSqlListModel();
qsm1->setStringList(qsl);
ui->lv->setModel(qsm1);

QDirModel* qdm=new QDirModel();
ui->tv->setModel(qdm);
ui->tv->setRootIndex(qdm->index("/home"));
```

The QSql module comes with three types of models—QSqlQueryModel, QSqlTableModel and QSqlRelationalTableModel. These SQL models are combined generally with the Table View. In this article we'll discuss QSqlQueryModel and QSqlTableModel.

QSqlTableModel

This is derived from QSqlQueryModel. It provides a model to display and edit the data that belongs to a single table in a particular database. We'll explain the creation of this model and some of its important members using the following example.

Create a Qt GUI project with the added module QSql, and design the interface as shown in Figure 2. A table with the name `student` and fields `rollno`, `sname` and `marks` is assumed in the `test` database of MySQL.

We'll use the following names for the various widgets:

- Labels – `lblTableName`, `lblSortOn`, `lblFilter`, `lblEditStrategy`
- Line Edits – `leTableName`, `leSortOn`, `leFilter`
- Radio Buttons – `rbAscending`, `rbDescending`
- Combo Box – `cbEditStrategy`
- Push Buttons – `pbPopulate`, `pbInsert`, `pbSubmitAll`, `pbRevertAll`, `pbShowDetails`
- Table View – `tvData`
- Text Edit – `teDetails`

Add the following members to the header file:

```
QSqlDatabase db;
QSqlTableModel* model;
```

Fill the constructor with the code as discussed in the previous article for connection establishment.

Here's the code for the `Populate` button:

```
1 model = new QSqlTableModel;
2 model->setTable(ui->leTableName->text());
3 int sortIndex=model->fieldIndex(ui->leSortOn->text());
```

```
4 if(ui->rbAscending->isChecked())
5     model->setSort(sortIndex,Qt::AscendingOrder);
6 else
7     model->setSort(sortIndex,Qt::DescendingOrder);
8 model->setFilter(ui->leFilter->text());
9 switch(ui->cbEditStrategy->currentIndex())
10 {
11     case 0: model->setEditStrategy(QSqlTableModel::OnFieldChange);
12     break;
13     case 1: model->setEditStrategy(QSqlTableModel::OnRowChange);
14     break;
15     case 2: model->setEditStrategy(QSqlTableModel::OnManualSubmit);
16     break;
17 }
18 model->select();
19 ui->tvData->setModel(model);
```

Here is the description for the above code:

- In Line 2, the `setTable` method links the model with the specified table. It takes the connection object as an optional second argument.
- In Line 3, `fieldIndex` returns an index for the given field name in the form of an integer.
- In Lines 5 and 7 the `setSort` method sorts the table data on a given column with a given order. (You should call `setSort` before population. The sort function can only be used when the model is populated with table data.)
- In line 8 `setFilter` takes the where clause of SQL query as string and filter records.
- In lines 11, 13 and 15 we're setting the edit strategy as `onFieldChange`, `onRowChange` or `Manual`.
- In line 18 the `select` function populates the model with data.
- In line 19 `setModel` combines model and view.

Code for `Insert`:

```
model->insertRow(model->rowCount(),1);
```

After the last row, one empty record will be added. Enter some data in the empty cells and submit the operation.

Code for `Submit All`:

```
model->submitAll()
```

This operation commits all pending operations, if the edit strategy uses the manual mode. We can use the `submit()` function instead to commit any changes on a per-field or per-row basis.

Code for `Revert All`:

```
model->revertAll()
```

This rolls back all changes. Instead, you can also use the `revertRow(rowNum)` to discard any changes in the specified row and `revert()` to rollback any changes in the current row.

Code for `Show Details`: